

Required readings

Khan (2021) – *Embracing Microservices Design* (ch. 2–3)

Khan laat zien dat mislukte microservices-initiatieven meestal teruggaan op een zwakke domeinmodellering, niet op verkeerde technologiekeuzes. Hij pleit voor Domain-Driven Design en duidelijke bounded contexts, omdat negeren daarvan leidt tot “chatty”, sterk gekoppelde services, governance-problemen en microservices die geen echte business-waarde toevoegen (Khan, 2021). Available at: <https://www.packtpub.com/en-us/product/embracing-microservices-design-9781801818384>.

Patnaik et al. (2023) – *SLR on Security API Recommendations*

Patnaik et al. analyseren 47 jaar onderzoek en concluderen dat veel beveiligingsproblemen ontstaan doordat security-API's moeilijk correct te gebruiken zijn, ondanks sterke onderliggende algoritmen. Zij groeperen aanbevelingen rond veilige defaults, minimale configuratie en duidelijke foutmeldingen, maar benadrukken dat veel richtlijnen nog onvoldoende empirisch met echte ontwikkelaars zijn getest (Patnaik et al., 2023). Available at: <https://dl.acm.org/doi/10.1145/3561383>.

Kholidy (2021) – *Autonomous Mitigation in CPS*

Kholidy introduceert een Autonomous Response Controller die IDS, risk assessment en machine learning combineert om cyber-physical systems continu te monitoren en automatisch mitigerende acties te kiezen. Simulaties tonen snellere detectie en lagere impact van aanvallen, maar de effectiviteit blijft afhankelijk van de nauwkeurigheid van modellen en de beheersing van false positives (Kholidy, 2021). Available at: <https://www.sciencedirect.com/science/article/abs/pii/S0167739X19320680>.

Python Enhancement Proposals (2001) – *PEP 8: Style Guide for Python Code*

PEP 8 definieert de standaardstijl voor Python-code (bijvoorbeeld snake_case, vier spaties inspringing, consistente whitespace) met als doel leesbaarheid en uniforme codebases. De richtlijn benadrukt dat code vooral door mensen wordt gelezen en dat bewuste, consistente stijlkeuzes samenwerking en onderhoud sterk vereenvoudigen (Python Enhancement Proposals, 2001). Available at: <https://peps.python.org/pep-0008/>.

Valente (2024) – *Software Engineering: A Modern Approach* (7.1 & 7.4)

Valente presenteert softwarearchitectuur als de high-level organisatie van een systeem en illustreert met patronen zoals layered architecture en MVC hoe verantwoordelijkheden en afhankelijkheden beheersbaar blijven. In de bespreking van microservices benadrukt hij dat voordelen zoals onafhankelijke deployment gepaard gaan met operationele complexiteit, waardoor microservices alleen passend zijn wanneer ze aantoonbaar bij het domein en de organisatie passen (Valente, 2024). Available at: <https://softengbook.org/>.

Gandhi, Richards & Ford (2024) – *Head First Software Architecture* (ch. 8 & 10)

Deze hoofdstukken leren lezers architectonisch denken via concrete scenario's en oefeningen rond trade-offs tussen onder meer performance, maintainability en security. De auteurs benadrukken dat geen enkele architectuur "perfect" is en dat professionele architectuur draait om het expliciet herkennen, analyseren en beargumenteren van noodzakelijke compromissen (Gandhi, Richards and Ford, 2024). Available at: <https://www.oreilly.com/library/view/head-first-software/9781098134341/>.